SIEWS+ 5.0

Smart Integrated Early Warning System Plus

Technical Documentation

Version 5.0

AI-Powered Safety Monitoring System for Upstream Oil & Gas Facilities

Contents

1

Project Overview

1.1 Introduction

SIEWS+ 5.0 (Smart Integrated Early Warning System Plus) is an advanced AI-powered safety monitoring system specifically designed for upstream oil and gas facilities, including offshore platforms, onshore processing areas, and industrial sites with high safety requirements.

The system provides real-time monitoring capabilities including:

- Human presence detection using computer vision
- Zone-based restriction management via polygon overlays
- Personal Protective Equipment (PPE) violation detection
- Automated shutdown signal triggering for high-risk zone breaches
- Real-time WhatsApp notifications to supervisors
- Web-based monitoring dashboard

1.2 Project Scope

SIEWS+ 5.0 addresses critical safety monitoring requirements in hazardous industrial environments:

1. **Real-time Person Detection**: Continuous monitoring of personnel presence using YOLOv8 deep learning models
2. **PPE Compliance Monitoring**: Automated detection of safety equipment (helmets, vests, belts) usage
3. **Zone Access Control**: Polygon-based restricted area management with dwell-time tracking

4. **Environmental Hazard Detection**: Identification of road damage, open holes, and safety cones

5. **Incident Documentation**: Automated snapshot capture and incident logging for compliance reporting

1.3 Technology Stack

Table 1.1: Technology Stack Overview

Layer	**Technology**	**Purpose**
Frontend	Next.js 14 (App Router)	Web Dashboard UI
Styling	Tailwind CSS 3.4	Responsive Design
Backend	Python FastAPI	REST API, Streaming
Detection	YOLOv8n (Ultralytics)	Multi-stage Object Detection
Streaming	OpenCV + MJPEG	Video Frame Processing
Database	SQLite + SQLAlchemy	Persistent Storage
Realtime	FastAPI WebSocket	Live Alert Push
Notification	Fonnte API	WhatsApp Alerts
Face Rec.	dlib face_recognition	Biometric Personnel ID
OCR	EasyOCR	Uniform Code Reading

1.4 System Architecture Overview

Figure 1.1: System Architecture Flow

The core data flow follows this sequence:

```
IP Camera → Backend (YOLOv8) → Zone Check (polygon) →
Alert + Shutdown → WhatsApp → Dashboard
```

1.5 Project Directory Structure

```
migas-siews/
 backend/
    main.py              # FastAPI entry point, all API endpoints
```

```
    stream.py            # StreamManager (detection pipeline)
    detector.py          # MultiStagePipeline (4-stage YOLO)
    polygon.py           # Ray-casting point-in-polygon
    zone_tracker.py     # PersonZoneTracker (dwell-time engine)
    violation_checker.py # ViolationChecker (PPE, zone, hazard)
    notifier.py          # WhatsApp via Fonnte API
    shutdown.py          # Shutdown signal handler
    models.py            # SQLAlchemy ORM models
    database.py          # SQLite engine, init_db()
    config.py            # Environment variables loader
    drawing.py           # OpenCV drawing utilities
    face_manager.py      # Face registration & recognition
    ocr_engine.py        # EasyOCR uniform code reader
    video_processor.py   # Async video upload processing
    detection_models.py # Class enums and thresholds
    requirements.txt    # Python dependencies
    .env                 # Environment configuration
    static/
        snapshots/        # Violation screenshots
        faces/           # Registered face images
        uploads/         # Uploaded video files
frontend/
    app/
       layout.tsx       # Root layout + Navbar
       page.tsx        # Redirects to /dashboard
       dashboard/      # Main monitoring page
       incidents/      # Incident log + filters
       zones/          # Zone management
       settings/      # System configuration
       analyze/       # Photo analysis page
       video-analysis/ # Video upload + processing
       faces/         # Personnel biometric registry
    components/
       Navbar.tsx      # Top navigation bar
       VideoCanvas.tsx # MJPEG stream + polygon overlay
       ZoneEditor.tsx  # Zone list panel
       AlertFeed.tsx   # Live WebSocket alert sidebar
```

```
      AlertCard.tsx   # Individual alert card
      ShutdownBanner.tsx # Emergency shutdown banner
      ImageTester.tsx # Image upload component
      LoadingScreen.tsx
   package.json
   tailwind.config.ts
training/
   train_stage3_fire_smoke_v2.ipynb
   train_stage3_fire_smoke.py
   requirements_train.txt
dataset/                 # Dataset storage
backend-go/              # Go-based backend (alternative)
camera-gateway/          # Camera gateway service
README.md
```

2

Backend Components

2.1 Main Application (main.py)

The `main.py` file serves as the FastAPI application entry point, handling all REST API endpoints, WebSocket connections, and CORS configuration.

2.1.1 API Endpoints

Method	Endpoint	Description
GET	`/stream`	MJPEG video stream with YOLO overlay
GET	`/polygons`	List all zones
POST	`/polygons`	Create a new zone
PUT	`/polygons/{id}`	Update zone
DELETE	`/polygons/{id}`	Delete zone + cascade records
GET	`/alerts`	Paginated incident log with filters
POST	`/alerts/{id}/resolve`	Mark incident resolved
POST	`/alerts/{id}/false-positive`	Mark as false positive
GET	`/alerts/{id}/detections`	Per-object detection crops
POST	`/shutdown/trigger`	Manual shutdown signal
GET	`/settings`	Get all settings
POST	`/settings`	Update a setting
POST	`/settings/notify-test`	Send test WhatsApp
WS	`/ws/alerts`	Real-time alert push WebSocket
WS	`/ws/camera`	Browser camera → AI → annotated frame
GET	`/stats`	Dashboard statistics
GET	`/analytics/compliance`	PPE compliance statistics
POST	`/ai/analyze-image`	Analyze uploaded image
POST	`/analyze/image`	Upload image for multi-stage detection

POST	`/stream/simulate`	Inject static image into live stream
POST	`/stream/reset`	Reset to live camera
POST	`/video/upload`	Upload video for async processing
GET	`/video/jobs`	List video processing jobs
GET	`/video/jobs/{id}`	Get job status
GET	`/video/jobs/{id}/result`	Get full detection results
GET	`/video/annotated/{id}`	Serve annotated video
DELETE	`/video/jobs/{id}`	Delete a job
GET	`/faces`	List registered faces
POST	`/faces/register`	Register a new face
DELETE	`/faces/{id}`	Delete a face
POST	`/faces/train`	Re-load face database
POST	`/ai/generate-report`	LLM incident report generation
POST	`/ai/schedule`	Agentic scheduling (mock)
GET	`/health`	Health check

2.2 Database Models (models.py)

The backend uses SQLAlchemy ORM for database management with SQLite as the database engine.

2.2.1 Zone Model

Represents restricted zone polygons on the monitoring area.

```
1  class Zone(Base):
2      __tablename__ = "zones"
3
4      id = Column(Integer, primary_key=True, index=True)
5      name = Column(String(100), nullable=False)
6      vertices_json = Column(Text, nullable=False)  # JSON: [[x,y],
           ...]
7      color = Column(String(7), default="#FF0000")
8      active = Column(Boolean, default=True)
9      risk_level = Column(String(10), default="high")  # "high" | "
           low"
10     zone_type = Column(String(20), default="restricted")
11     dwell_threshold_sec = Column(Integer, default=10)
```

```
12      created_at = Column(DateTime, default=datetime.utcnow)
```

2.2.2 Alert Model

Incident log entry for detected violations.

```
1  class Alert(Base):
2      __tablename__ = "alerts"
3
4      id = Column(Integer, primary_key=True, index=True)
5      zone_id = Column(Integer, ForeignKey("zones.id"), nullable=True
           )
6      confidence = Column(Float, default=0.0)
7      snapshot_path = Column(String(255), nullable=True)
8      timestamp = Column(DateTime, default=datetime.utcnow)
9      shutdown_triggered = Column(Boolean, default=False)
10     resolved = Column(Boolean, default=False)
11     violation_type = Column(String(50), nullable=True)
12     false_positive = Column(Boolean, default=False)
13     ppe_detail = Column(Text, nullable=True)  # JSON
14     person_name = Column(String(100), nullable=True)
15     uniform_code = Column(String(50), nullable=True)
```

2.2.3 DetectionLog Model

Per-object crop for false positive review and audit.

```
1  class DetectionLog(Base):
2      __tablename__ = "detection_logs"
3
4      id = Column(Integer, primary_key=True, index=True)
5      alert_id = Column(Integer, ForeignKey("alerts.id"), nullable=
           True)
6      class_name = Column(String(50), nullable=False)
7      confidence = Column(Float, default=0.0)
8      crop_path = Column(String(255), nullable=True)
9      frame_number = Column(Integer, nullable=True)
10     bbox_json = Column(Text, nullable=True)  # JSON: {x1,y1,x2,y2}
11     is_false_positive = Column(Boolean, default=False)
12     created_at = Column(DateTime, default=datetime.utcnow)
```

2.2.4 Other Models

- **ShutdownLog**: Records shutdown events with trigger source (auto/manual)
- **Setting**: Key-value configuration store for system parameters
- **VideoJob**: Async video processing job tracking
- **ZoneOccupancy**: Zone dwell audit log for compliance reporting

2.3 Core Detection Pipeline

2.3.1 StreamManager (stream.py)

The StreamManager orchestrates the entire detection pipeline:

1. Captures frames from IP camera at configured interval
2. Runs multi-stage YOLO detection pipeline
3. Checks for PPE violations using ViolationChecker
4. Checks for zone violations using polygon algorithms
5. Triggers alerts and shutdown signals when violations detected
6. Generates MJPEG stream with overlay annotations
7. Broadcasts alerts via WebSocket to connected dashboards

2.3.2 MultiStagePipeline (detector.py)

Implements a 4-stage detection pipeline:

Stage	Model	Classes
1	yolov8n.pt	person
2	best_stage2_labeled_safety.pt	unknown, belt, helmet, vest
3	best_stage3_openhole.pt	barricade, hard_hat, safety_cone, open_hole, vest_env
4	best_jalan_berlubang.pt	lubang, retak, tambalan

2.4 Polygon Algorithms (polygon.py)

2.4.1 Ray-Casting Point-in-Polygon

The system uses the ray-casting algorithm to determine if a point is inside a polygon:

```
1  def point_in_polygon(point: Tuple[float, float], polygon: List[List
       [float]]) -> bool:
2      """Ray-casting␣algorithm␣to␣determine␣if␣a␣point␣is␣inside␣a␣
           polygon.
3  ␣␣␣␣All␣coordinates␣are␣normalized␣(0.0␣-␣1.0)."""
4      x, y = point
5      n = len(polygon)
6      inside = False
7
8      j = n - 1
9      for i in range(n):
10         xi, yi = polygon[i]
11         xj, yj = polygon[j]
12
13         if ((yi > y) != (yj > y)) and (x < (xj - xi) * (y - yi) / (
               yj - yi) + xi):
14             inside = not inside
15         j = i
16
17     return inside
```

All polygon coordinates are normalized (0.0 to 1.0) for resolution independence.

2.5 Violation Checking (violation_checker.py)

The ViolationChecker validates detected objects against safety rules:

2.5.1 PPE Violations

- NO_HELMET: Person detected without helmet
- NO_VEST: Person detected without safety vest
- NO_BELT: Person detected without safety belt

2.5.2 Zone Violations

- Checks person (head, chest, feet points) inside restricted zones
- Triggers shutdown for high-risk zones
- Logs warning for low-risk zones

2.5.3 Hazard Violations

- Person center overlapping environmental hazard (open hole, road damage)
- Generates critical alert with snapshot

2.6 Person Tracking (zone_tracker.py)

The PersonZoneTracker maintains stable person identities across frames:

- Uses centroid proximity matching for track association
- Generates unique 8-character hex UUIDs for track IDs
- Tracks dwell time in zones with configurable thresholds
- Emits events: zone_entry, dwell_warning, dwell_critical, zone_exit

2.7 Notification System (notifier.py)

WhatsApp notifications via Fonnte API:

```
def send_whatsapp(message: str, recipient: str) -> dict:
    """Send single WhatsApp message via Fonnte API."""
    url = "https://fonnte.com/api/send_message.php"
    data = {
        "token": FONNTE_TOKEN,
        "target": recipient,
        "message": message,
    }
    response = requests.post(url, data=data)
    return response.json()
```

2.8 Face Recognition (face_manager.py)

- Uses dlib face_recognition (128-dim HOG encoding)
- OpenCV Haar cascade fallback when dlib unavailable
- Registers faces with name, code, and phone number
- Stores encodings in face_db.json for persistence
- Singleton instance: face_manager

2.9 OCR Engine (ocr_engine.py)

- Uses EasyOCR for uniform code reading from person chest area
- Applies CLAHE, denoising, and sharpening preprocessing
- Fuzzy-matches against registered codes from face database
- Graceful fallback when EasyOCR unavailable

3

Frontend Components

3.1 Next.js Application Structure

The frontend is built with Next.js 14 App Router with Tailwind CSS for styling.

3.1.1 Page Routes

Page	Path	Purpose
Root	`/`	Redirects to `/dashboard`
Dashboard	`/dashboard`	Main monitoring - live stream, zone editor, alerts
Zones	`/zones`	Zone management without canvas drawing
Incidents	`/incidents`	Historical alert log with CSV export
Settings	`/settings`	Camera, inference, WhatsApp configuration
Analyze Photo	`/analyze`	Single image upload + detection analysis
Video Analysis	`/video-analysis`	Video upload, job list, per-frame results
Faces	`/faces`	Personnel biometric registry

3.2 Key Components

3.2.1 VideoCanvas.tsx

The primary monitoring component that displays the MJPEG stream with polygon overlay capabilities:

Features:

- MJPEG stream display from `/stream` endpoint
- Canvas overlay for zone polygon drawing

- Click-to-place vertices, double-click to close polygon
- Alert flash animation (red border + overlay text)
- Camera offline overlay with reconnection logic
- LIVE indicator with ping animation

VideoCanvas Interface:

```
interface ZoneData {
  id: number;
  name: string;
  vertices: number[][];  // Normalized [x, y] coordinates
  color: string;
  active: boolean;
  risk_level: string;
}
```

3.2.2 ZoneEditor.tsx

Zone management panel with the following features:

- Lists all zones with active/inactive toggle
- "Tambah Zona" button to start drawing mode
- Delete zone button
- Risk level badge display
- Vertex count display

3.2.3 AlertFeed.tsx

Real-time alert sidebar component:

- WebSocket connection to `/ws/alerts`
- Real-time alert list (max 30, newest first)
- Audio beep on new alerts (oscillator-based, different tone for high/low risk)
- Resolve and False Positive actions
- Connection status indicator

3.2.4 Navbar.tsx

Fixed navigation bar with:

- Logo and navigation links
- Active link highlighting (amber color)
- Real-time stats chips: camera status, zone count, today's alerts
- Unresolved alert badge on "Incidents" link

3.2.5 ShutdownBanner.tsx

Emergency shutdown alert display:

- Full-width emergency shutdown banner
- Displays zone name and timestamp (WIB timezone)
- "Confirm & Reset" button to resolve alert

3.3 State Management

The frontend uses React's built-in state management with hooks:

- `useState` for local component state
- `useEffect` for side effects (WebSocket, polling)
- `useCallback` for memoized callbacks
- API calls to FastAPI backend via `NEXT_PUBLIC_API_URL`

3.4 Styling

All styling is done with Tailwind CSS utility classes. Custom theme colors:

- Primary: Amber (`amber-500`)
- Danger: Red (`red-600`)
- Success: Green (`green-500`)
- Info: Blue (`blue-500`)

4

Database Schema

4.1 Entity Relationship Diagram

The database consists of 7 primary tables with the following relationships:

- `alerts` references `zones` (zone_id)
- `detection_logs` references `alerts` (alert_id)
- `shutdown_logs` references `zones` (zone_id)
- `video_jobs` is standalone
- `zone_occupancy` references `zones` (zone_id)
- `settings` is a key-value store

4.2 Table Definitions

4.2.1 zones

Column	Type	Default	Description
id	INTEGER	PK	Primary key
name	VARCHAR(100)	NOT NULL	Zone display name
vertices_json	TEXT	NOT NULL	JSON list of normalized [x,y] pairs
color	VARCHAR(7)	#FF0000	Hex color code
active	BOOLEAN	TRUE	Zone enabled/disabled
risk_level	VARCHAR(10)	high	"high" or "low"
zone_type	VARCHAR(20)	restricted	Zone classification
dwell_threshold_sec	INTEGER	10	Dwell time before alert
created_at	DATETIME	now()	Creation timestamp

4.2.2 alerts

Column	Type	Default	Description
id	INTEGER	PK	Primary key
zone_id	INTEGER	FK	Reference to zones.id
confidence	FLOAT	0.0	Detection confidence
snapshot_path	VARCHAR(255)	NULL	Path to violation screenshot
timestamp	DATETIME	now()	Alert timestamp
shutdown_triggered	BOOLEAN	FALSE	Shutdown signal sent
resolved	BOOLEAN	FALSE	Alert resolved status
violation_type	VARCHAR(50)	NULL	Type of violation detected
false_positive	BOOLEAN	FALSE	Marked as false positive
ppe_detail	TEXT	NULL	JSON with PPE details
person_name	VARCHAR(100)	NULL	Recognized person name
uniform_code	VARCHAR(50)	NULL	OCR-detected uniform code

4.2.3 detection_logs

Column	Type	Default	Description
id	INTEGER	PK	Primary key
alert_id	INTEGER	FK	Reference to alerts.id
class_name	VARCHAR(50)	NOT NULL	Detected class name
confidence	FLOAT	0.0	Detection confidence
crop_path	VARCHAR(255)	NULL	Path to cropped image
frame_number	INTEGER	NULL	Source video frame number
bbox_json	TEXT	NULL	JSON bounding box coords
is_false_positive	BOOLEAN	FALSE	Marked as false positive
created_at	DATETIME	now()	Creation timestamp

4.2.4 shutdown_logs

Column	Type	Default	Description
id	INTEGER	PK	Primary key
zone_id	INTEGER	FK	Reference to zones.id
trigger_source	VARCHAR(20)	NOT NULL	"auto" or "manual"
triggered_at	DATETIME	now()	Shutdown timestamp

4.2.5 settings

Column	Type	Default	Description
id	INTEGER	PK	Primary key
key	VARCHAR(100)	UNIQUE	Setting key name
value	TEXT	NULL	Setting value

4.2.6 video_jobs

Column	Type	Default	Description
id	INTEGER	PK	Primary key
filename	VARCHAR(255)	NOT NULL	Original filename
file_path	VARCHAR(255)	NOT NULL	Stored file path
status	VARCHAR(20)	pending	pending/processing/done/failed
progress	INTEGER	0	Progress percentage 0-100
total_frames	INTEGER	0	Total frames in video
processed_frames	INTEGER	0	Frames processed so far
result_json	TEXT	NULL	Full detection results JSON
annotated_video_path	VARCHAR(255)	NULL	Path to output video
error_message	TEXT	NULL	Error description if failed
created_at	DATETIME	now()	Job creation time
completed_at	DATETIME	NULL	Job completion time

4.2.7 zone_occupancy

Column	Type	Default	Description
id	INTEGER	PK	Primary key
zone_id	INTEGER	FK	Reference to zones.id
track_id	VARCHAR(20)	NOT NULL	8-char hex UUID for person
entry_time	DATETIME	NOT NULL	Zone entry timestamp
exit_time	DATETIME	NULL	Zone exit timestamp
dwell_sec	INTEGER	NULL	Total dwell duration
alert_level	VARCHAR(20)	none	none/warning/critical

5

Detection Models and Classes

5.1 Detection Class Definitions (detection_models.py)

5.1.1 PPE Classes (Stage 2)

Index	Name	Description
0	UNKNOWN	Unknown object class
1	BELT	Safety belt/buckle
2	HELMET	Safety helmet/hard hat
3	VEST	Safety vest

5.1.2 Environment Classes (Stage 3)

Index	Name	Description
0	BARRICADE	Safety barricade
1	HARD_HAT	Worker wearing hard hat
2	SAFETY_CONE	Safety cone marker
3	OPEN_HOLE	Open hole/excavation
4	VEST_ENV	Worker wearing safety vest (env)

5.1.3 Road Damage Classes (Stage 4)

Index	Name	Description
0	LUBANG	Pothole
1	RETAK	Crack
2	TAMBALAN	Patch/filled area

5.2 Confidence Thresholds

Detection Type	Threshold	Description
PERSON_CONFIDENCE_THRESHOLD	0.50	Minimum person detection confidence
PPE_CONFIDENCE_THRESHOLD	0.35	Minimum PPE detection confidence
ENV_CONFIDENCE_THRESHOLD	0.40	Minimum environment hazard confide
ROAD_CONFIDENCE_THRESHOLD	0.40	Minimum road damage confidence
SAFETY_CONE_CONFIDENCE	0.50	Minimum safety cone confidence
PPE_IOU_THRESHOLD	0.10	IoU threshold for PPE-person assignm

5.3 Model Files

Model File	Stage	Classes
best_stage2_labeled_safety.pt	2 (PPE)	unknown, belt, helmet, vest
best_stage3_openhole.pt	3 (Env)	barricade, hard_hat, safety_cone, open_hole, vest_env
best_jalan_berlubang.pt	4 (Road)	lubang, retak, tambalan
stage2_ppe_harness.pt	2 (PPE)	harness-specific
stage3_environment.pt	3 (Env)	environment-specific
stage4_infrastructure.pt	4 (Infra)	infrastructure-specific

5.4 Training Configuration

Training uses YOLOv8n as the base model with the following hyperparameters:

Parameter	Value	Type	Description
epochs	80	int	Training epochs
batch	8-16	int	Batch size (auto-adjusted for GPU VRAM)
imgsz	640	int	Input image size
patience	15	int	Early stopping patience
freeze	10	int	Frozen layers (transfer learning)
optimizer	AdamW	string	Optimizer type
lr0	0.001	float	Initial learning rate
lrf	0.01	float	Final learning rate (cosine decay)
hsv_h	0.03	float	Hue augmentation
hsv_s	0.9	float	Saturation augmentation

hsv_v	0.5	float	Value/brightness augmentation
mosaic	1.0	float	4-image mosaic augmentation
mixup	0.1	float	Mixup augmentation probability
flipud	0.1	float	Vertical flip probability
degrees	5.0	float	Rotation augmentation
scale	0.6	float	Scale augmentation
erasing	0.3	float	Random erasing probability

6

Configuration

6.1 Backend Environment (.env)

```
CAMERA_SOURCE=0
FACILITY_NAME=Offshore Platform A
CONFIDENCE_THRESHOLD=0.3
DETECTION_INTERVAL=3
NOTIFY_COOLDOWN=300
FONNTE_TOKEN=
DEFAULT_RECIPIENTS=
```

6.2 Backend Requirements (requirements.txt)

```
fastapi==0.115.0
uvicorn==0.30.0
opencv-python==4.10.0.84
ultralytics==8.4.37
sqlalchemy==2.0.35
aiofiles==24.1.0
python-dotenv==1.0.1
httpx==0.27.2
python-multipart==0.0.9
websockets==12.0
face-recognition==1.3.0
easyocr==1.7.0
dlib==19.24.0
numpy>=1.24.0
pillow>=10.0.0
```

6.3 Frontend Configuration (package.json)

```
{
  "name": "siews-frontend",
  "version": "5.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "next": "14.2.35",
    "react": "^18",
    "react-dom": "^18"
  },
  "devDependencies": {
    "@types/node": "^20",
    "@types/react": "^18",
    "tailwindcss": "^3.4.1",
    "typescript": "^5"
  }
}
```

6.4 Tailwind Configuration (tailwind.config.ts)

```
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: "#fffbeb",
          100: "#fef3c7",
          200: "#fde68a",
          300: "#fcd34d",
          400: "#fbbf24",
          500: "#f59e0b",
          600: "#d97706",
          700: "#b45309",
          800: "#92400e",
          900: "#78350f",
        },
      },
```

```
25      },
26    },
27    plugins: [],
28  };
29  export default config;
```

7

Installation and Setup

7.1 System Requirements

- Python 3.10+
- Node.js 18+
- 8GB+ RAM (16GB recommended for training)
- NVIDIA GPU with 6GB+ VRAM (for detection)
- Internet connection for API services

7.2 Backend Setup

7.2.1 Clone and Install

```
# Clone repository
git clone https://github.com/username/migas-siews.git
cd migas-siews/backend

# Create virtual environment
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# Install dependencies
pip install -r requirements.txt

# Configure environment
cp .env.example .env
```

```
# Edit .env with your configuration
```

7.2.2 Run Backend

```
# Development
uvicorn main:app --reload --host 0.0.0.0 --port 8001

# Production
uvicorn main:app --host 0.0.0.0 --port 8001 --workers 4
```

7.3 Frontend Setup

7.3.1 Install and Run

```
# Navigate to frontend
cd ../frontend

# Install dependencies
npm install

# Run development server
npm run dev

# Build for production
npm run build
npm start
```

7.4 Dataset Preparation

7.4.1 Dataset Structure

```
dataset/
 Fire and Smoke.v14i.yolo26.zip
 jalan berlubang.v3-tambahan-dataset.yolo26/
 stage3/
    train/
       images/
       labels/
```

```
    val/
       images/
       labels/
    test/
        images/
        labels/
stage3_environment.yaml
```

7.4.2 YAML Configuration

```yaml
# stage3_fire_smoke.yaml
path: /path/to/dataset/stage3
train: train/images
val: val/images
test: test/images

nc: 2
names: ['fire', 'smoke']
```

7.5 Training

7.5.1 Local Training

```
cd training
python train_stage3_fire_smoke.py
```

7.5.2 Kaggle Training

1. Upload training notebook to Kaggle
2. Attach dataset from `/kaggle/input`
3. Enable GPU P100/T4
4. Run All cells
5. Download `best_stage3_fire_smoke.pt` from Output

7.6 Model Deployment

```
# Copy trained model to backend/models/
cp best_stage3_fire_smoke.pt ../backend/models/

# Restart backend to load new model
uvicorn main:app --reload
```

8

API Reference

8.1 REST Endpoints

8.1.1 Zone Management

GET /polygons

Retrieve all zones.

Response:

```
{
  "zones": [
    {
      "id": 1,
      "name": "Restricted Area A",
      "vertices": [[0.1, 0.2], [0.3, 0.4], ...],
      "color": "#FF0000",
      "active": true,
      "risk_level": "high",
      "zone_type": "restricted",
      "dwell_threshold_sec": 10
    }
  ]
}
```

POST /polygons

Create a new zone.

Request Body:

```
{
  "name": "Restricted Area A",
  "vertices": [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]],
  "color": "#FF0000",
  "risk_level": "high",
  "zone_type": "restricted",
  "dwell_threshold_sec": 10
}
```

8.1.2 Alert Management

GET /alerts

Retrieve paginated alerts.

Query Parameters:

- `page`: Page number (default: 1)
- `limit`: Items per page (default: 20)
- `resolved`: Filter by resolved status
- `violation_type`: Filter by violation type
- `zone_id`: Filter by zone ID
- `start_date`: Filter by start date
- `end_date`: Filter by end date

Response:

```
{
  "alerts": [...],
  "total": 100,
  "page": 1,
  "limit": 20
}
```

8.2 WebSocket Endpoints

8.2.1 /ws/alerts

Real-time alert push to connected clients.

Server Message:

```
{
  "type": "alert",
  "data": {
    "id": 123,
    "zone_name": "Restricted Area A",
    "violation_type": "ZONE_VIOLATION",
    "confidence": 0.85,
    "timestamp": "2024-01-15T10:30:00Z",
    "snapshot_path": "/static/snapshots/alert_123.jpg",
    "shutdown_triggered": true
  }
}
```

8.2.2 /ws/camera

Browser camera → AI → annotated frame pipeline.

Client sends: Base64 encoded frame **Server receives:** Annotated frame with bounding boxes

8.3 Detection Classes Reference

8.3.1 PPE Detection Classes

Class ID	Name	Associated Violation
0	UNKNOWN	None
1	BELT	NO_BELT
2	HELMET	NO_HELMET
3	VEST	NO_VEST

8.3.2 Violation Types

Violation Type	Severity	Action
NO_HELMET	High	Alert + Log
NO_VEST	High	Alert + Log
NO_BELT	Medium	Alert + Log
ZONE_VIOLATION	High	Alert + Shutdown + WhatsApp
DWELL_WARNING	Medium	Alert + Log
DWELL_CRITICAL	High	Alert + Shutdown + WhatsApp
HAZARD_PROXIMITY	High	Alert + Log

9

Troubleshooting

9.1 Common Issues

9.1.1 Camera Not Displaying

Symptoms: VideoCanvas shows "Camera Offline" overlay.
Solutions:

1. Check camera source configuration in `.env`
2. Verify IP camera is accessible and streaming
3. Check if MJPEG stream endpoint responds: `GET /stream`
4. Restart backend service

9.1.2 Detection Not Working

Symptoms: No bounding boxes on stream, no person detection.
Solutions:

1. Verify YOLO model files exist in `backend/models/`
2. Check model filenames match configuration in `detector.py`
3. Ensure GPU is available and CUDA is installed
4. Check confidence thresholds are appropriate
5. Review backend logs for detection errors

9.1.3 WhatsApp Notifications Not Sending

Symptoms: Alerts logged but no WhatsApp message received.
Solutions:

1. Verify FONNTE_TOKEN is set correctly in `.env`
2. Check DEFAULT_RECIPIENTS contains valid phone numbers
3. Test notification manually via `POST /settings/notify-test`
4. Check Fonnte API status and quota

9.1.4 Zone Polygon Drawing Not Working

Symptoms: Clicking on canvas does not place vertices.
Solutions:

1. Ensure "Tambah Zona" drawing mode is activated
2. Check browser console for JavaScript errors
3. Verify WebSocket connection is established
4. Clear browser cache and reload

9.1.5 Training Fails

Symptoms: Training notebook errors or poor mAP results.
Solutions:

1. Verify dataset structure is correct (train/val/images + labels)
2. Check label files for correct class IDs (0=fire, 1=smoke)
3. Ensure YAML `names` is a LIST not a dict
4. Adjust batch size if GPU OOM errors occur
5. Verify YOLOv8n model downloads successfully

9.2 Performance Optimization

9.2.1 GPU Memory Issues

If encountering CUDA out of memory errors:

- Reduce batch size to 8 or 4
- Reduce image size to 416 or 320
- Process frames at lower frequency (increase DETECTION_INTERVAL)
- Disable unnecessary detection stages

9.2.2 Slow Inference

If detection is too slow:

- Use YOLOv8n (nano) instead of larger models
- Enable detection interval (process every N frames)
- Reduce MJPEG stream quality
- Limit number of active zones

9.3 Log Locations

Component	**Log Location**
Backend FastAPI	stdout/stderr
Frontend Next.js	Browser console
Model Training	`training/runs/stage3_fire_smoke/`
Snapshots	`backend/static/snapshots/`
Video Jobs	Database video_jobs table

A

Glossary

Term	**Definition**
SIEWS	Smart Integrated Early Warning System
PPE	Personal Protective Equipment (helmet, vest, belt)
MJPEG	Motion JPEG - video streaming format
IoU	Intersection over Union - bounding box overlap metric
YOLO	You Only Look Once - object detection architecture
OCR	Optical Character Recognition - text reading from images
WebSocket	Bidirectional real-time communication protocol
ROI	Region of Interest
FPS	Frames Per Second
UUID	Universal Unique Identifier
Fonnte	WhatsApp API service provider

B

Class Enumerations

B.1 PPEClass

```
class PPEClass(IntEnum):
    UNKNOWN = 0
    BELT = 1
    HELMET = 2
    VEST = 3
```

B.2 EnvClass

```
class EnvClass(IntEnum):
    BARRICADE = 0
    HARD_HAT = 1
    SAFETY_CONE = 2
    OPEN_HOLE = 3
    VEST_ENV = 4
```

B.3 RoadClass

```
class RoadClass(IntEnum):
    LUBANG = 0     # Pothole
    RETAK = 1      # Crack
    TAMBALAN = 2   # Patch
```

C

File Inventory

C.1 Backend Files

File	Lines	Purpose
main.py	800	FastAPI app, all endpoints
stream.py	400	StreamManager class
detector.py	300	MultiStagePipeline
polygon.py	100	Ray-casting algorithm
zone_tracker.py	250	PersonZoneTracker
violation_checker.py	200	ViolationChecker
notifier.py	100	WhatsApp integration
shutdown.py	50	Relay trigger
models.py	200	SQLAlchemy models
database.py	50	DB connection
face_manager.py	300	Face recognition
ocr_engine.py	150	OCR processing
detection_models.py	100	Class enums

C.2 Frontend Pages

Page	Location	Purpose
Dashboard	app/dashboard/page.tsx	Main monitoring interface
Incidents	app/incidents/page.tsx	Alert history and export
Zones	app/zones/page.tsx	Zone management
Settings	app/settings/page.tsx	System configuration
Analyze	app/analyze/page.tsx	Image analysis tool
Video Analysis	app/video-analysis/page.tsx	Video processing

Faces	app/faces/page.tsx	Personnel registry

Bibliography

bibitemyolov8 Ultralytics, "YOLOv8 Documentation," 2024. [Online]. Available: https://docs.ultralytics.com/ bibitemfastapi FastAPI, "FastAPI Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/ bibitemnextjs Vercel, "Next.js Documentation," 2024. [Online]. Available: https://nextjs.org/docs bibitemroboflow Roboflow, "Roboflow Universe," 2024. [Online]. Available: https://universe.roboflow.com/ bibitemfonnte Fonnte, "Fonnte WhatsApp API," 2024. [Online]. Available: https://fonnte.com/ bibitemdlib Davis King, "dlib Documentation," 2024. [Online]. Available: http://dlib.net/ bibitemeasyocr JaidedAI, "EasyOCR Documentation," 2024. [Online]. Available: https://www.jaided.ai/easyocr/